Introduction to Structured Prompts with PromptKit

Bringing software engineering discipline to AI prompts

Alan Jowett

The Most Important Code You're Not Engineering

Every AI-assisted task — investigating bugs, writing requirements,

reviewing code — lives or dies by the prompt that drives it

Yet most teams still write these prompts ad hoc:

- Copy-pasted from chat histories
- Untested — no way to detect regressions
- Inconsistent — every engineer writes their own
- Impossible to improve systematically

Your code is reviewed, tested, and version-controlled.
Your prompts? Probably pasted from a Slack thread.

Why Ad-Hoc Prompting Fails at Scale

Non-deterministic — same prompt, different results every time;

LLMs are inherently stochastic

Not version-controlled — prompts live in chat histories,

Slack threads, sticky notes

Not testable — no way to know if a prompt "regressed"

after you changed it

Not reusable — every engineer writes their own from scratch

Not composable — can't mix and match proven reasoning patterns

What If Prompts Were Engineered Artifacts?

Version-controlled in Git — diffs, blame, history, branching

Modular — compose proven components instead of writing from scratch

Testable — compare outputs against known-good reference prompts

Shareable — team-wide library of best practices, not tribal knowledge

Improvable — systematic iteration, not random tweaking

This is what PromptKit does.

Part 1
How PromptKit Works

Five Composable Layers

TEMPLATE	The task itself — "Investigate this bug..."
FORMAT	Output structure — investigation report, design doc...
TAXONOMY	Classification — D1–D16 spec drift, K1–K14 kernel defects
PROTOCOL	How to reason — anti-hallucination, root-cause analysis
PERSONA	Who the LLM is — systems engineer, security auditor...

Each layer is a separate Markdown file — compose by reference, not by copy-paste

Layer 1 — Personas: Who the LLM Is

15 built-in personas across 6 engineering domains

Software: systems-engineer · security-auditor · software-architect
Hardware: electrical-engineer · embedded-firmware-engineer · rf-engineer · mechanical-engineer
Specialty: protocol-architect · specification-analyst · reverse-engineer · test-engineer

Each defines: domain expertise, tone, behavioral constraints, reasoning style

Or define your own custom persona inline

Layer 2 — Protocols: How the LLM Reasons

48 protocols in three categories:

Guardrails (5) — Cross-cutting safety

anti-hallucination · self-verification · adversarial-falsification · minimal-edit-discipline · operational-constraints

Analysis (17) — Domain-specific checks

memory-safety-c · thread-safety · security-vulnerability · schematic-compliance · kernel-correctness · and more...

Reasoning (26) — Systematic methodologies

root-cause-analysis · requirements-elicitation · traceability-audit · protocol-evolution · change-propagation · and more...

Each is a multi-phase, step-by-step methodology — protocols compose additively

Layers 3 & 4 — Formats and Taxonomies

21 output formats — structured document templates:

investigation-report · requirements-doc · design-doc · validation-plan · structured-patch · presentation-kit · agent-instructions · and more...

5 classification taxonomies:

specification-drift (D1–D16): Cross-document divergence

kernel-defect-categories (K1–K14): OS kernel/driver defects

stack-lifetime-hazards (H1–H5): Memory escape hazards

cpp-review-patterns (19 labels): C++ code review findings

protocol-change-categories (PC1–PC8): Protocol changes

Fun fact: this presentation was built with PromptKit's author-presentation template + presentation-kit format!

Layer 5 — Templates: The Task Itself

```yaml
---
name: investigate-bug
persona: systems-engineer
protocols:
  - guardrails/anti-hallucination
  - guardrails/self-verification
  - guardrails/operational-constraints
  - reasoning/root-cause-analysis
taxonomies:
  - stack-lifetime-hazards
format: investigation-report
params:
  problem_description: "What is the bug?"
  code_context: "Relevant source code"
  environment: "OS, compiler, runtime"
---
```

64 templates — each composes persona + protocols + format via YAML frontmatter.
Templates declare input/output contracts → enables chaining into pipelines.

How Components Snap Together

```
# Identity
You are a senior systems engineer with deep expertise
in memory management, concurrency, and debugging ...

# Reasoning Protocols
## Protocol: Anti-Hallucination Guardrails
Rule 1: Every claim must be KNOWN, INFERRED, or ASSUMED ...

## Protocol: Root Cause Analysis
Phase 1: Characterize the symptom precisely ...
Phase 2: Generate ≥ 3 hypotheses before investigating ...

# Output Format
Output MUST contain: Executive Summary, Findings,
Root Cause, Remediation, Prevention ...

# Task
Investigate: segfault on startup under high connection load ...
```

Components are included verbatim — no summarization. One coherent Markdown document.

Part 2
Protocol Spotlights

Spotlight: Anti-Hallucination Guardrails

Epistemic Labeling — every claim must be categorized:

- KNOWN: Directly stated in the provided context
- INFERRED: Reasonable conclusion with explicit reasoning chain
- ASSUMED: Not established — flagged with [ASSUMPTION]

Refusal to Fabricate — no invented function names, API signatures, version numbers. Use [UNKNOWN] placeholders instead.

Uncertainty Disclosure — enumerate multiple interpretations, don't choose one silently

Source Attribution — cite where information came from ("per requirements doc, section 3.2")

Hard trigger: if ASSUMED content exceeds ~30%, stop and request more context

Spotlight: Root-Cause Analysis Protocol

Phase 1 — Symptom Characterization

Describe precisely, establish timeline, determine blast radius

Phase 2 — Hypothesis Generation

Generate >= 3 hypotheses before investigating ANY of them

Include one "non-obvious" hypothesis. No anchoring on first guess.

Phase 3 — Evidence Gathering & Elimination

For each hypothesis: minimal investigation to confirm/eliminate. Classify: CONFIRMED / ELIMINATED / INCONCLUSIVE

Phase 4 — Root Cause Identification

Distinguish root cause from proximate cause. Trace full causal chain. "If we fix only the proximate cause, will the root cause produce other failures?"

Phase 5 — Remediation: Fix the root cause, not the symptom. Propose secondary fixes.

Part 3
What Can You Do With PromptKit?

157 Components Across 6 Engineering Domains

Domain	What You Can Do
Software	Code review, bug investigation, requirements, design docs, testing, refactoring
Hardware / EE	Schematic review, BOM audit, PCB layout, simulation validation, power budgets
Embedded Firmware	Boot sequences, OTA updates, flash management, power-fail-safe, watchdogs
Protocol Engineering	RFC authoring, protocol evolution, conflict analysis, validation specs
Specification Analysis	Invariant extraction, traceability audits, interface contracts, spec diffing
DevOps & CI/CD	Pipelines, issue triage, PR triage, releases, infrastructure review

The Engineering Lifecycle: Bootstrap → Evolve → Maintain

BOOTSTRAP	Scan any repo, extract requirements + design + validation specs
EVOLVE	Propagate changes through specs & implementation with adversarial audits
MAINTAIN	Periodic drift detection — find where code and specs have diverged

Each is an interactive workflow — runs in your LLM session with built-in challenge and verification

Part 4
Case Study — Finding a Crypto Vulnerability via Spec Review

Case Study: IoT Protocol Crypto Migration

Project: Sonde — a production IoT runtime

260+ requirements across 15 specification documents

Task: Replace gateway's asymmetric key-pair pairing encryption with AES-256-GCM using a pre-shared key (PSK)

Impact: Every radio frame in the protocol is affected

Approach: PromptKit's engineering-workflow — 8-phase incremental development with adversarial audits at each transition

Phase 1 discovery: The initial understanding was fundamentally wrong — wrong crypto model, scope significantly underestimated

Nonce Reuse Vulnerability Found by Automated Spec Review

Round 5 of spec review: automated reviewer identified that GCM nonce construction could allow nonce reuse between request/response pairs using the same PSK

Original nonce construction:

gcm_nonce = SHA-256(psk)[0..3] || frame_nonce[8]

Problem: A request and response could use the same nonce value — catastrophic for GCM security

Fix: Include msg_type in nonce construction:

gcm_nonce = SHA-256(psk)[0..3] || msg_type || frame_nonce[8]

This vulnerability survived 6 rounds of human review. The structured, adversarial prompt caught it.

What the Case Study Tells Us

Full migration: 28 modified, 16 retired, 8 new requirements across 9 specification documents

PromptKit's contribution:

- Caught a genuine cryptographic vulnerability that human review missed
- Forced systematic exploration — no anchoring on first impressions
- Adversarial auditing at each phase prevented compounding errors

Human review still mattered:

- Phase 7 caught a frame ownership error — only domain knowledge could catch it
- Survived all automated review

Lesson: Structured prompts + human judgment > either alone

Part 5
Live Demo

Demo: Bootstrapping a Prompt with PromptKit

We'll use npx @alan-jowett/promptkit to launch an interactive session

The bootstrap engine reads manifest.yaml, discovers all 157 components

We'll select a task, provide parameters, and watch it assemble a structured prompt

Watch for:

- Persona selection
- Protocol composition
- Format enforcement
- Parameter substitution

Demo Fallback: What the Assembled Prompt Looks Like

```
# Identity
You are a senior systems engineer with deep expertise in
memory management, concurrency, performance, and debugging.
You reason from first principles ...

# Reasoning Protocols
## Anti-Hallucination Guardrails
Every claim MUST be categorized as KNOWN, INFERRED, or ASSUMED.
Do NOT invent function names, API signatures ...

## Root Cause Analysis
Phase 1: Describe the symptom precisely ...
Phase 2: Generate ≥ 3 hypotheses before investigating ...

# Output Format
Findings, Root Cause, Remediation, Prevention ...

# Task
Investigate: {{problem_description}}
```

Assembled prompt — all components included verbatim, parameters filled in

Part 6
Getting Started

Three Ways to Use PromptKit

1. npx (recommended — no clone needed):
   npx @alan-jowett/promptkit
2. Clone the repo — full access to all components:
   git clone https://github.com/microsoft/promptkit.git
   copilot -i "Read and execute bootstrap.md"
3. Manual — paste bootstrap.md + manifest.yaml into any LLM session

Works with: GitHub Copilot CLI · Claude Code · Any LLM

Persistent Agent Instructions — Version-Control Your Team's AI Practices

PromptKit produces persistent instruction files for:

GitHub Copilot: .github/instructions/*.instructions.md — composable skill files with applyTo glob targeting

Claude Code: CLAUDE.md

Cursor: .cursorrules

Example: "Always review C code for memory safety" → persistent skill file applied to every *.c and *.h file

Commit to your repo → every engineer gets the same AI behavior automatically

Key Takeaways

1. Prompts are code — treat them with the same engineering rigor as your software
2. Compose, don't copy-paste — modular components beat ad-hoc prompts every time
3. Structured reasoning bounds non-determinism — protocols like anti-hallucination and root-cause analysis make LLM outputs more reliable
4. Test against references — compare outputs against known-good prompts to catch regressions
5. Humans + structured prompts > either alone — the crypto case study proves it

PromptKit

Get Started with PromptKit

🔗 aka.ms/PromptKit

📦 npx @alan-jowett/promptkit

📄 MIT License — open source

🐙 github.com/microsoft/promptkit

157 components · 15 personas · 48 protocols · 21 formats · 64 templates

Thank you! Questions?